1 marca 2026
Tomasz Koźmiński
Adam Kuryś
Filip Zawadzki

Opiekun zespołu projektowego:
dr inż. Przemysław Korpas

AOSYW

Projekt Oczy - etap 2

Politechnika Warszawska

WYDZIAŁ ELEKTRONIKI
I TECHNIK INFORMACYJNYCH

Spis treści

1 Architektura oprogramowania i założenia projektowe **2**
1.1 Schemat blokowy systemu . . . 2
1.2 Działanie aplikacji generującej obraz . . . 2
1.2.1 Wątek 1 . . . 2
1.2.2 Wątek 2 . . . 3

2 Konfiguracja systemu FreeRTOS **5**
2.1 Synchronizacja . . . 5

3 Implementacja modułu czujnika odległości TOF **5**
3.1 Opis struktur danych . . . 5
3.1.1 Struktura sterownika: RangeSensor_t . . . 5
3.1.2 Struktura danych pomiarowych: RangeSensorFrame . . . 6
3.1.3 Typ wyliczeniowy: RangeSensor_Status . . . 6
3.1.4 Struktura konfiguracyjna: ObjectTrackerConfig . . . 7
3.1.5 Struktura główna algorytmu: ObjectTracker . . . 7
3.1.6 Struktura wynikowa: TrackedObject . . . 8
3.1.7 Struktury pomocnicze (Prywatne) . . . 8
3.2 Opis funkcji modułu RangeSensor (Adapter Sprzętowy) . . . 9
3.3 Opis funkcji modułu ObjectTracker . . . 9
3.3.1 Funkcje prywatne (Algorytm Analityczny) . . . 9
3.3.2 Funkcje Publiczne (API) . . . 10

4 Implementacja modułu ekranu LED **11**
4.1 Działanie matrycy . . . 11
4.2 Frame . . . 11
4.3 LedMatrix . . . 14
4.4 Image . . . 16
4.4.1 ImageBackground . . . 16
4.4.2 ImageMovingItem . . . 17
4.5 Font . . . 18
4.6 Canvas . . . 20

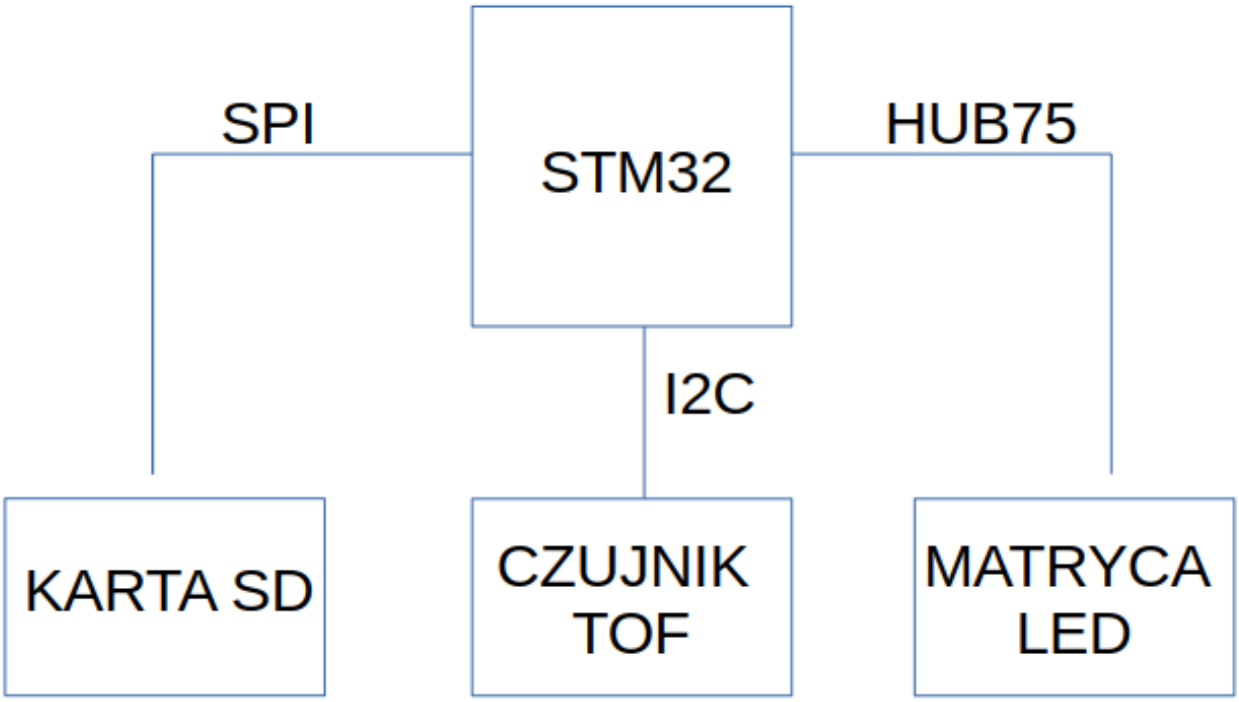
5 Implementacja modułu pamięci SD **21**
5.1 Warstwa sprzętowa: SD_SPI Driver . . . 21
5.1.1 Struktury danych . . . 21
5.1.2 Zastosowane wzorce projektowe . . . 21
5.1.3 Funkcje publiczne . . . 21
5.2 Warstwa systemu plików: FatFS Integration . . . 22
5.2.1 Architektura . . . 22
5.2.2 Uwaga o osDelay() . . . 23
5.3 Warstwa czytnika animacji: AnimationReader (OOP w C) . . . 23
5.3.1 Struktury danych . . . 23
5.3.2 Format pliku animacji . . . 24
5.3.3 Funkcje publiczne . . . 24
5.3.4 Uwagi implementacyjne . . . 25
5.4 Warstwa mapowania: AnimationMapper . . . 25
5.4.1 Koncepcja siatki animacji . . . 25
5.4.2 Dwie metody mapowania . . . 25
5.4.3 Diagram przepływu . . . 26
5.5 Integracja całości w FreeRTOS . . . 26
5.5.1 Inicjalizacja . . . 26
5.5.2 Konfiguracja zasobów . . . 27
5.5.3 Kodowanie błędów . . . 28
5.6 Podsumowanie architektury . . . 28

1 Architektura oprogramowania i założenia projektowe

1.1 Schemat blokowy systemu

Odpowiedzialność projektowa:

- Moduł pamięci SD - Adam Kuryś
- Moduł ekranu LED - Filip Zawadzki
- Moduł czujnika odległości TOF - Tomasz Koźmiński



Rysunek 1: Schemat blokowy i protokołów komunikacji

1.2 Działanie aplikacji generującej obraz

Główna aplikacja składa się z dwóch wątków utworzonych za pomocą FreeRTOS. Obiekt LedMatrix jest inicjalizowany w funkcji main przed rozpoczęciem działania wątków FreeRTOS.

1.2.1 Wątek 1

Listing 1: funkcja wątku 1

```
void LoopTask(void const * argument)
{
  for(;;){
    LedMatrix_DisplayFrame();
    osDelay(1);
  }
}
```

Jeden wątek z wysokim priorytetem, wyświetla obraz na matrycy za pomocą obiektu singleton LedMatrix. Wysoki priorytet jest niezbędny w celu uzyskania płynnego obrazu ze stałą jasnością pikseli w rzędach. Wątek zajmuje się jedynie odświeżaniem obrazu. Po wyświetleniu jednej klatki wątek wprowadza opóźnienie 1ms gdzie zwalnia zasoby dla drugiego wątku

1.2.2 Wątek 2

Listing 2: funkcja wątku 2

```
void StartDefaultTask(void const * argument)
{
  /* USER CODE BEGIN 5 */

  // TOF sersor's variables
  RangeSensor mySensor;
  ObjectTracker myTracker;
  RangeSensorFrame currentFrame;

  ObjectTrackerConfig tracker_cfg = {
      .min_dist_mm     = 50.0f,
      .max_dist_mm     = 2000.0f,
      .fade_dist_mm    = 300.0f,
      .min_mass_detect = 3.5f,
      .smooth_factor   = 0.15f,
      .sensor_fov_deg  = 45.0f  // Standardowe FOV dla VL53L7CX (45x45)
  };


  // Image's variables
  Canvas mainCanvas;
  Frame outputFrame;
  ImageBackground mainBG;
  ImageMovingItem mainItem;

  MoveRange moveLimts = {
    .maxOffsetX = ItemOffsets[imageIndex][0],
    .maxOffsetY = ItemOffsets[imageIndex][1]
  };

  PositionDataRange inputRange = {
    .minX = 0.0,
    .maxX = 7.0,
    .minY = 0.0,
    .maxY = 7.0
  };

  Image mainImage = {
    .Background = &mainBG,
    .MovingItem = &mainItem
  };


  // TOF init
  VL53L7CX_Adapter_Construct(&mySensor, &hi2c2, 0x52);

  if (RangeSensor_Init(&mySensor) != RS_OK){
    Error_Handler();
  }

  ObjectTracker_Init(&myTracker, &tracker_cfg);

```

```
  if (RangeSensor_Start(&mySensor) != RS_OK) {
    Error_Handler();
  }

  // Init Canvas and Image
  if(Canvas_Create(&mainCanvas)){
    Error_Handler();
  }
  ImageBackground_Init(&mainBG, BackgroundLibrary[imageIndex], WHITE);
  ImageMovingItem_Init(&mainItem, MovingItemLibrary[imageIndex], moveLimts,
      inputRange, RED);

  // task loop
  for(;;)
  {
      // read sensor
      RangeSensor_Status status = RangeSensor_GetFrame(&mySensor, &currentFrame);

      if (status == RS_OK) {
          // calculate sensor read
          ObjectTracker_Process(&myTracker, &currentFrame);

          // if button pressed load new image
          if(changeImage){
            changeImage = false;
            ImageBackground_SetNewBackgroundData(&mainBG, BackgroundLibrary[
                imageIndex % LIB_IMAGES_CNT]);
            ImageMovingItem_SetNewItemData(&mainItem, MovingItemLibrary[imageIndex %
                LIB_IMAGES_CNT]);
          }

          // set image accordingly to sensor read
          ImageMovingItem_SetPosition(&mainItem, ObjectTracker_GetX(&myTracker),
              ObjectTracker_GetY(&myTracker));
          // put image on canvas
          Canvas_PutImage(&mainCanvas, &mainImage);

          // generate output frame
          outputFrame = Canvas_GenerateFrame(&mainCanvas);
          // send frame to display
          LedMatrix_ChangeFrame(outputFrame);
      }
      else if (status == RS_ERROR_HW_FAILURE) {
          Error_Handler();
      }
  }
  /* USER CODE END 5 */
}
```

Drugi wątek z niższym priorytetem zajmuje się generowaniem obrazu na podstawie odczytów z czujnika TOF oraz podrzuceniem wygenerowanego obrazu do LedMatrix. Praca wątku zaczyna się od konfiguracji i inicjalizacji czujnika TOF oraz stworzenia płótna na generowany obraz oraz przypisania do niego obrazu. Następnie zaczyna się pętla gdzie jest odczyt z czujnika TOF na którego podstawie zmieniane jest położenie dynamicznego elementu obrazu a następnie generowana jest klatka która jest podrzucana do obiektu LedMatrix. Wątek 2 blokuje wątek 1 przy podmianie klatki w celu zapewnienia, że dostęp do dzielonej przez wątki pamięci ma w danym momencie tylko jeden wątek. Dodatkowo aplikacja pozwala na zmianę wyświetlanych obrazów za pomocą przycisku na płytce, czego obsługa jest realizowana za pomocą przerwania na pinie z przyciskiem oraz wykrycia zmiany w pętli tego wątku.

2 Konfiguracja systemu FreeRTOS

Najkorzystniejszym wykorzystaniem RTOS w przypadku naszego projektu byłoby użycie scheduling oparty na Time Slicingu pomiędzy wątkami, pozwoliłoby to nam jasno określić czas przeznaczony na każdy z wątków, gdzie obydwa pracowałyby na takim samym priorytecie. Jednak jako, że FreeRTOS w przypadku wątków o równym priorytecie działa jedynie jako RoundRobin, czyli przydziela równe przedziały czasowe między wątki, niezbędne było danie wyższego priorytetu wątkowi odpowiedzialnemu za wyświetlanie obrazu. Potrzeba taka wynika z faktu że w trakcie wyświetlania obrazu czas podświetlenia każdego rzędu matrycy musi być taki sam aby uniknąć różnicy w jasności i migotań.

2.1 Synchronizacja

- W celu uniknięcia przekłamań pamięci dane dzielone i używane przez obydwa wątki są zabezpieczone mutexem
- W celach synchronizacji transferów DMA z działaniem procesora użyto TaskNotify z przerwania
- W celu zwolnienia zasobów dla wątku o niższym priorytecie użyto TaskDelay
- Niektóre klasy posiadają własne implementacje zabezpieczenia w postaci mutex'u na potrzebę ich możliwego użytku w scenariuszu wielowątkowym

3 Implementacja modułu czujnika odległości TOF

Implementacja systemu opiera się na wydzieleniu niezależnych **bloków funkcjonalnych**, co pozwala na przejrzysty podział zadań w kodzie. Moduł składa się z dwóch głównych elementów: `RangeSensor` (obsługa czujnika) oraz `ObjectTracker` (analiza danych). Poniżej przedstawiono szczegółowy opis implementacji tych bloków, zastosowane wzorce projektowe oraz sposób przekazywania danych między nimi.

3.1 Opis struktur danych

Poniżej przedstawiono kluczowe struktury danych wykorzystane w projekcie, wraz z analizą ich roli w architekturze systemu.

3.1.1 Struktura sterownika: RangeSensor_t

Jest to główna struktura reprezentująca instancję czujnika w warstwie sterownika urządzenia. Pełni ona rolę adaptera, ukrywając szczegóły techniczne biblioteki producenta przed logiką aplikacji.

Listing 3: Definicja struktury RangeSensor_t

```
struct RangeSensor_t {
    /* 1. Hardware Proxy Instance
     * Glowna struktura sterownika ST. Zawiera konfiguracje,
     * kalibracje oraz strukture 'platform' (adres I2C). */
    VL53L7CX_Configuration dev;

    /* 2. Bufor Posredni (Proxy Output)
     * Biblioteka ST zwraca dane w bardzo rozbudowanej strukturze.
     * Bufor ten przechowuje surowe wyniki przed konwersja. */
    VL53L7CX_ResultsData st_results;

    /* 3. Stan Adaptera */
    uint8_t is_ranging;          // 1 = pomiar aktywny, 0 = stop
    uint8_t initialization_done; // Zabezpieczenie przed uzyciem bez Init
};
```

Opis pól:

- `dev`: Jest to struktura dostarczana przez bibliotekę producenta (ST ULD). Przechowuje niskopoziomowe ustawienia czujnika, adresy I2C oraz bufory komunikacyjne. Pełni rolę "uchwytu"do fizycznego urządzenia.
- `st_results`: Bufor pamięci alokowany statycznie wewnątrz obiektu. Biblioteka VL53L7CX zwraca wyniki w bardzo dużej strukturze (zawierającej m.in. dane o temperaturze, sygnale, szumie). Przechowujemy ją tutaj, aby uniknąć alokacji na stosie przy każdym odczycie.
- `is_ranging / initialization_done`: Flagi stanu, które zapobiegają wywołaniu funkcji pomiarowych przed inicjalizacją lub ponownemu uruchomieniu już działającego czujnika.

Zastosowane wzorce:

- **Adapter Pattern**: Struktura ta "owija"natywne, skomplikowane struktury producenta (`VL53L7CX_Configuration`), udostępniając wyższym warstwom uproszczony interfejs zarządzania.
- **State Flag**: Wykorzystanie zmiennych statusowych do prostej kontroli przepływu (zapobieganie błędom sekwencji wywołań).

3.1.2 Struktura danych pomiarowych: RangeSensorFrame

Struktura ta pełni rolę DTO (Data Transfer Object) pomiędzy warstwą sprzętową a logiczną.

Listing 4: Definicja struktury RangeSensorFrame

```
typedef struct {
    uint16_t distances_mm[64]; // Macierz odleglosci (8x8 stref)
    uint8_t  statuses[64];     // Statusy wiarygodnosci pomiaru
    bool     is_valid;         // Flaga poprawnosci ramki
} RangeSensorFrame;
```

Opis pól:

- `distances_mm`: Spłaszczona tablica 2D reprezentująca macierz 8x8. Zawiera odległości w milimetrach.
- `statuses`: Tablica kodów błędów dla każdej strefy (zgodnie z dokumentacją ST: 5 i 9 to pomiary poprawne).
- `is_valid`: Flaga logiczna określająca ważność całej ramki danych. Jeśli ma wartość `false`, oznacza to, że odczyt z czujnika się nie powiódł (np. brak nowych danych lub błąd I2C) i algorytm powinien pominąć takie dane.

Zastosowane wzorce:

- **Data Transfer Object (DTO)**: Struktura ta nie zawiera żadnej logiki (metod), służy wyłącznie do transportu sformatowanych danych. Pozwala to całkowicie uniezależnić algorytm `ObjectTracker` od biblioteki sprzętowej ST.

3.1.3 Typ wyliczeniowy: RangeSensor_Status

Typ ten definiuje zbiór kodów powrotu używanych przez wszystkie funkcje modułu `RangeSensor`. Pozwala to na ustandaryzowaną obsługę błędów w całym systemie.

Listing 5: Definicja enum RangeSensor_Status

```
typedef enum {
    RS_OK = 0,                    // Sukces
    RS_ERROR_SENSOR_INIT    = -1, // Blad inicjalizacji
    RS_ERROR_HW_FAILURE     = -2, // Blad sprzetowy (I2C)
    RS_ERROR_TIMEOUT        = -3, // Przekroczenie czasu oczekiwania
    RS_ERROR_DATA_NOT_READY = -4, // Brak nowych danych (polling)
    RS_ERROR_PARAM          = -5, // Bledny parametr wejsciowy
    RS_ERROR_DATA_CORRUPTED = -6  // Dane niespojne
} RangeSensor_Status;
```

Zastosowane wzorce:

- **Error Code / Status Return**: Każda funkcja biblioteki zwraca status wykonania. Wartość 0 (`RS_OK`) zawsze oznacza sukces, natomiast wartości ujemne oznaczają konkretne kategorie błędów, co ułatwia debugowanie i implementację mechanizmów ponawiania (retry mechanism).

3.1.4 Struktura konfiguracyjna: ObjectTrackerConfig

Struktura przechowująca parametry strojące algorytmu śledzenia.

Listing 6: Definicja struktury ObjectTrackerConfig

```
typedef struct {
    float min_dist_mm;        // Dolny prog odciecia (filtr pasmowy)
    float max_dist_mm;        // Gorny prog odciecia (tlo)
    float fade_dist_mm;       // Strefa miekkiego przejscia (okno trapezowe)
    float min_mass_detect;    // Minimalna suma wag wymagana do detekcji
    float smooth_factor;      // Wspolczynnik alfa filtru EMA
    float sensor_fov_deg;     // Kat widzenia czujnika (do obliczen geometrycznych)
} ObjectTrackerConfig;
```

Zastosowane wzorce:

- **Immutable (Configuration) Object**: Oddzielenie konfiguracji od stanu pozwala na łatwe strojenie systemu bez ryzyka przypadkowej modyfikacji parametrów w trakcie działania programu.

3.1.5 Struktura główna algorytmu: ObjectTracker

Główny obiekt (kontekst) modułu analitycznego.

Listing 7: Definicja struktury ObjectTracker

```
typedef struct {
    /* Wynik dzialania algorytmu */
    TrackedObject target;

    /* Stan wewnetrzny (pamiec filtrow) */
    float _prev_x;
    float _prev_y;
    float _prev_dist;
    float _prev_prob;

    /* Wskaznik do konfiguracji */
    const ObjectTrackerConfig *config;
} ObjectTracker;
```

Opis pól:

- `target`: Zagnieżdżona struktura zawierająca aktualnie wyliczoną pozycję, odległość i pewność detekcji. Jest to wyjście algorytmu.
- `_prev_...`: Zmienne „prywatne” (zwyczajowo poprzedzone podkreśleniem), przechowujące stan filtrów z poprzedniego kroku (wymagane do działania filtru IIR/EMA).
- `config`: Wskaźnik (referencja) do obiektu konfiguracyjnego w pamięci Flash.

Zastosowane wzorce:

- **Context Object Pattern**: Struktura ta gromadzi w jednym miejscu cały stan niezbędny do działania algorytmu (historię filtrów, konfigurację, wyniki). Jest ona przekazywana do funkcji jako wskaźnik `self`. Dzięki temu funkcje przetwarzające są bezstanowe (nie używają zmiennych `static` wewnątrz siebie ani zmiennych globalnych), co ułatwia testowanie i pozwala na ewentualne uruchomienie wielu instancji trackera równolegle.

3.1.6 Struktura wynikowa: TrackedObject

Jest to struktura wyjściowa modułu `ObjectTracker`. Reprezentuje ona ostateczny, przetworzony rezultat działania algorytmu, gotowy do wykorzystania przez warstwę aplikacji (np. wyświetlenie na ekranie).

Listing 8: Definicja struktury TrackedObject

```
typedef struct {
    bool  is_detected;  // Czy wykryto obiekt?
    float probability;  // Pewnosc detekcji (0.0 - 1.0)
    float position_x;   // Pozycja X (0.0 - 7.0)
    float position_y;   // Pozycja Y (0.0 - 7.0)
    float angle_deg;    // Kat odchylenia w stopniach
    float distance_mm;  // Odleglosc do srodka ciezkosci
} TrackedObject;
```

Opis pól:

- `is_detected`: Flaga binarna. Pozwala na szybkie sprawdzenie, czy należy reagować na obiekt, bez analizowania prawdopodobieństwa.
- `probability`: Znormalizowana wartość (0.0 - 1.0) określająca „jakość” sygnału. Może być użyta do wizualizacji paska pewności.
- `position_x /y`: Współrzędne „sub-pikselowe”. Mimo że czujnik ma rozdzielczość 8x8, dzięki zastosowaniu średniej ważonej, struktura zwraca pozycję jako liczbę zmiennoprzecinkową (np. 3.5), co pozwala na płynniejsze śledzenie ruchu.
- `angle_deg`: Wartość wyliczona geometrycznie na podstawie pola widzenia (FOV) czujnika i pozycji X.

Zastosowane wzorce:

- **Data Transfer Object (DTO)**: Struktura ta służy wyłącznie do przekazywania danych wynikowych. Jest całkowicie oddzielona od logiki wewnętrznej trackera i jego konfiguracji.

3.1.7 Struktury pomocnicze (Prywatne)

W celu zachowania czystości kodu i separacji odpowiedzialności, wewnątrz pliku implementacyjnego `ObjectTracker.c` zdefiniowano strukturę pomocniczą, niewidoczną w publicznym API (brak definicji w pliku .h).

Listing 9: Prywatna struktura analityczna (ObjectTracker.c)

```
typedef struct {
    float sum_weight;       // Calkowita masa (suma wag)
    float sum_weight_x;     // Akumulator dla pozycji X
    float sum_weight_y;     // Akumulator dla pozycji Y
    float sum_weight_dist;  // Akumulator dla odleglosci
} AnalysisData;
```

Rola w systemie: Struktura ta służy do przekazywania wyników pośrednich między kolejnymi krokami algorytmu. Funkcja analizująca obraz (*Step 1*) zwraca wypełnioną strukturę `AnalysisData`, która następnie jest przekazywana do funkcji decyzyjnej i aktualizującej.

Zastosowane wzorce:

- **Private Data Structure**: Ukrycie tej struktury w pliku .c (zamiast w nagłówku) to przykład silnej enkapsulacji. Użytkownik biblioteki nie musi wiedzieć o istnieniu pośrednich wyników obliczeń wagowych – interesuje go tylko wynik końcowy (`TrackedObject`).

3.2 Opis funkcji modułu RangeSensor (Adapter Sprzętowy)

Moduł ten odpowiada za bezpośrednią komunikację z układem VL53L7CX poprzez interfejs I2C.

1. **Konstruktor:** `VL53L7CX_Adapter_Construct`
   - **Argumenty:** Wskaźnik do `RangeSensor`, uchwyt I2C (`hi2c`), adres urządzenia.
   - **Działanie:** Realizuje wstrzykiwanie zależności (Dependency Injection). Przypisuje konkretny interfejs sprzętowy STM32 (np. `&hi2c1`) do struktury sterownika, nie uruchamiając jeszcze komunikacji.
2. **Inicjalizacja:** `RangeSensor_Init`
   - **Zwraca:** Status operacji (`RangeSensor_Status`).
   - **Działanie:** Sekwencja startowa wymagana przez układ:
     (a) Sprawdzenie komunikacji (Ping/IsAlive).
     (b) **Ładowanie Firmware:** Przesłanie binary bloba do pamięci RAM czujnika (funkcja `vl53l7cx_init`). Jest to proces czasochłonny, ale niezbędny dla czujników TOF tej generacji.
     (c) Konfiguracja parametrów pracy: ustawienie rozdzielczości 8x8 stref, trybu ciągłego oraz częstotliwości próbkowania (15 Hz).
3. **Sterowanie:** `RangeSensor_Start / Stop`
   Funkcje te służą do włączania i wyłączania silnika pomiarowego w czujniku. `Start` uruchamia ciągłe generowanie przerwań/danych przez czujnik.
4. **Pobieranie danych:** `RangeSensor_GetFrame`
   - **Argumenty:** Wskaźnik do czujnika, wskaźnik do struktury wyjściowej `out_frame`.
   - **Zwraca:** `RS_OK` (jeśli są nowe dane) lub błąd/brak danych.
   - **Działanie:** Funkcja realizuje odczyt w trybie Polling (odpytywanie):
     (a) Pyta czujnik, czy nowe dane są gotowe (`vl53l7cx_check_data_ready`).
     (b) Jeśli tak, pobiera dużą strukturę wyników z czujnika do bufora wewnętrznego.
     (c) Przepisuje (mapuje) kluczowe dane – odległość i status – do lekkiej struktury `RangeSensorFrame`, która jest zrozumiała dla reszty systemu.

3.3 Opis funkcji modułu ObjectTracker

Moduł ten realizuje logikę przetwarzania obrazu z czujnika TOF. Funkcje zostały podzielone na prywatne (pomocnicze, realizujące kroki algorytmu) oraz publiczne (API).

3.3.1 Funkcje prywatne (Algorytm Analityczny)

1. **Funkcja ważąca:** `_calculate_pixel_weight`
   Jest to fundamentalna funkcja dla jakości działania trackera. Określa ona „wiarygodność” pojedynczego punktu pomiarowego.
   - **Argumenty:** Wskaźnik do obiektu `ObjectTracker`, odległość `dist_mm`, status strefy `status`.
   - **Zwraca:** Wartość `float` z zakresu $0.0 - 1.0$ (gdzie 1.0 to pewny obiekt).
   - **Działanie:** Funkcja implementuje tzw. **Okno Trapezowe** (Trapezoidal Windowing).
     (a) *Filtracja błędów:* Jeśli status czujnika jest inny niż 5 lub 9 (poprawne pomiary), waga wynosi 0.
     (b) *Odcięcie tła:* Jeśli odległość jest poza zakresem `min_dist - max_dist`, waga wynosi 0.
     (c) *Strefa przejściowa (Fade):* Aby uniknąć nagłych skoków wartości, gdy obiekt wchodzi w pole widzenia, zastosowano liniowe narastanie wagi w strefach brzegowych (zdefiniowanych przez `fade_dist_mm`).

2. **Filtr wygładzający:** `_low_pass_filter`
   - **Argumenty:** Wartość bieżąca, wartość poprzednia (cel), współczynnik α.
   - **Zwraca:** Przefiltrowana wartość `float`.
   - **Działanie:** Implementuje cyfrowy filtr dolnoprzepustowy IIR (Exponential Moving Average) wg wzoru:
   $$Y_{out} = Y_{in} \cdot (1 - \alpha) + Y_{prev} \cdot \alpha$$
   Pozwala to na wygładzenie szumu pomiarowego i płynniejsze śledzenie ruchu obiektu.
3. **Krok 1 - Analiza:** `_step1_analyze_frame`
   - **Argumenty:** Instancja trackera, ramka danych wejściowych.
   - **Zwraca:** Strukturę `AnalysisData` z sumami ważonymi.
   - **Działanie:** Funkcja iteruje przez wszystkie 64 strefy czujnika (macierz 8x8). Dla każdej strefy oblicza jej wagę. Jeśli waga jest niezerowa, funkcja akumuluje (dodaje) wartości do sum częściowych:
     - `sum_weight`: Całkowita „masa” widocznego obiektu.
     - `sum_weight_x /y/dist`: Iloczyn pozycji strefy i jej wagi (moment siły).

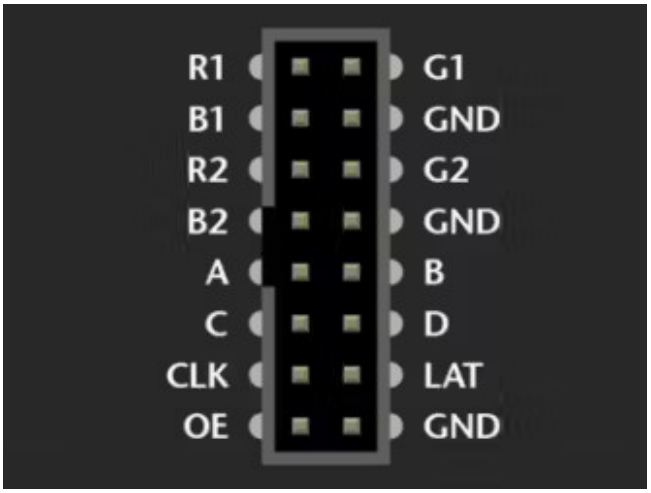
   Dane te są niezbędne do późniejszego wyznaczenia środka ciężkości.
4. **Krok 2 - Decyzja:** `_step2_check_detection`
   - **Argumenty:** Instancja trackera, obliczona całkowita masa (`total_mass`).
   - **Zwraca:** `bool` (true = obiekt wykryty).
   - **Działanie:** Proste progowanie (Thresholding). Sprawdza, czy suma wag pikseli przekracza skonfigurowany próg `min_mass_detect`. Zapobiega to detekcji szumu jako obiektu.
5. **Krok 3 - Aktualizacja:** `_step3_update_state`
   - **Argumenty:** Instancja trackera, dane z analizy, flaga detekcji.
   - **Działanie:** Jeśli obiekt został wykryty, funkcja oblicza jego współrzędne metodą **Środka Ciężkości**:
   $$Pos_X = \frac{\sum(Waga_i \cdot X_i)}{\sum Waga_i}$$
   Pozwala to na uzyskanie rozdzielczości „sub-pikselowej” (np. pozycja 3.5). Następnie surowe wyniki są przepuszczane przez filtr dolnoprzepustowy (`_low_pass_filter`) i zapisywane w publicznej strukturze `target`. Funkcja wylicza również kąt odchylenia obiektu na podstawie geometrii sensora.

3.3.2 Funkcje Publiczne (API)

1. `ObjectTracker_Init`
   Inicjalizuje strukturę, zerując pamięć i przypisując wskaźnik do konfiguracji. Ustawia filtry na wartości początkowe (środek zakresu), aby uniknąć gwałtownego przeskoku przy pierwszym pomiarze.
2. `ObjectTracker_Process`
   Główna funkcja. Wywołuje sekwencyjnie kroki 1, 2 oraz 3. Jest jedyną funkcją, którą musi wywoływać użytkownik w głównej pętli programu, aby zaktualizować stan algorytmu.
3. **Gettery (np.** `ObjectTracker_GetX`**)**
   Zestaw funkcji dostępowych zwracających przeliczone wartości (X, Y, Dystans, Kąt, Prawdopodobieństwo) z wewnętrznej struktury `target`. Zapewniają hermetyzację danych.

4 Implementacja modułu ekranu LED

4.1 Działanie matrycy



Rysunek 2: Złącze HUB75

Matryca sterowana jest za pomocą złącza HUB75, które udostępnia nam piny:

- **RGB1/RGB2** - odpowiadające za wybór koloru
- **A, B, C, D** - odpowiadające za wybór rzędu (16 rzędów, 2 jednocześnie, RGB1 i RGB2 opisują pixele tych dwóch rzędów)
- **CLK** - przesuwa rejestr przesuwny o jeden
- **LAT** - zatrzask zapisujący wartość kolorów dla rzędu
- **OE** - Output Enable włączający LEDy w rzędzie

Przebieg wyświetlania:

- wybranie rzędu za pomocą A,B,C i D
- ustawienie pinów RGB1 i RGB2 dla pixeli w dwóch obecnie wybranych rzędach oraz "uderzenie"CLK
- powtórzyć dla wszystkich 64 pixeli w rzędzie
- "uderzenie"LAT w celu zapisania stanu rejestru z kolorami pixeli
- włączenie OE w celu podświetlenia rzędu pixeli
- powtórzyć dla każdego rzędu w celu wyświetlenia całej klatki

Taki sposób sterowania matrycą wymusza bardzo szybkie oraz stałe odświeżanie ekranu w celu wyświetlania pełnych klatek ponieważ jednocześnie może być podświetlony tylko jeden rząd pixeli.

4.2 Frame

Format danych w którym przetrzymywane są dane gotowe bezpośrednio do wyświetlenia na matrycy. W celu wydajnego wyświetlenia danych użyty został rejestr BSRR portu C na którym znajdują się wszystkie piny obsługujące matrycę. Rejestr BSRR służy do atomowego ustawiania i zerowania wyjść GPIO. Dzięki niemu można zmieniać stan pojedynczych pinów bez ryzyka zakłóceń. Bity 0-15 ustawiają odpowiedni pin w stan wysoki, a bity 16-31 zerują wskazany pin. Rejestr ten nie przetrzymuje obecnego stanu pinów, tylko pozwala nam na zmianę ich stanu. Wykorzystując go możliwe jest jednoczesne włączenie/wyłączenie wszystkich pinów

jak i nie podjęcie żadnej akcji (pisząc do rejestu wartość 0). Dane typu Frame to wskaźniki na zaalokowany obszar pamięci przetrzymujący stany rejestru BSRR których kolejne pisanie do tego rejestru utworzy obraz na matrycy.

Dodatkowo do utworzenia koloru więcej niż 3 bitowego (RGB) użyto Bit Code Modulation. BCM to technika sterowania jasnością (a pośrednio kolorem RGB), która – podobnie jak PWM – wykorzystuje modulację czasową, ale zamiast stałej częstotliwości używa wag binarnych bitów. W naszym przypadku stworzyliśmy kolor 12 bitowy RGB444.

Listing 10: typ Frame oraz jego alokacja

```
typedef uint32_t * Frame;

//...

Frame Frame_Create(void)
{
    Frame newFrame = calloc(sizeof(uint32_t), ADDRESABLE_PIXELS * BCM_BIT_DEPTH);
    return newFrame;
}
```

Alokowany obszar wynika z :

- sizeof(uint32_t) - rejestr BSRR jest 32 bitowy wiec dana o jego stanie bedzie zapisywana w zmiennej typu uint32_t
- ADDRESABLE_PIXELS - ilość "pixeli"które jesteśmy w stanie ustawić, pomimo faktu, że matryca posiada 2048 (32 * 64) pixeli, tak naprawdę jedna instrukcja BSRR ustawia dwa piksele jednocześnie z odpowiednio kolorami RGB1 i RGB2, więc ADDRESABLE_PIXELS w naszym przypadku wynosi 1024
- BCM_BIT_DEPTH - iloraz nadprodukcji danych w celu stworzenia koloru

Założenia

- Każdy kanał (R, G, B) ma **4 bity** → zakres **0–15**
- Wagi bitów: **1, 2, 4, 8**
- Czas bazowy: **T**
- Czasy przedziałów BCM: **T, 2T, 4T, 8T**

Wybrany kolor (RGB444)

ini Copy code

```ini
R = 1010₂ = 10
G = 0111₂ = 7
B = 0011₂ = 3
```

Przedziały czasowe BCM

Bit	Czas	R	G	B
bit0	T	0	1	1
bit1	2T	1	1	1
bit2	4T	0	1	0
bit3	8T	1	0	0

Jak oko „widzi” kolor

Sumaryczny czas świecenia:

- **R = 2T + 8T = 10T**
- **G = T + 2T + 4T = 7T**
- **B = T + 2T = 3T**

Proporcje czasu świecenia odpowiadają dokładnie wartościom **RGB444 = (10, 7, 3)**.

Rysunek 3: Przykład z wyjaśnieniem działania BCM - wygenerowane z pomocą ChatGPT

Aby zastosować BCM jedną klatkę tak naprawdę trzeba wyświetlić cztery razy z 4 różnymi opóźnieniami, które stworzą widoczny dla oka kolor 12 bitowy.

4.3 LedMatrix

Listing 11: struktura klasy LedMatrix

```
// obiekt matrycy
typedef struct{
    SemaphoreHandle_t Mutex;
    // buffer na wywwietlane dane
    Frame DisplayData;
    // drugi buffer przyjmujqcy nowe dane
    Frame Buffer;
}LedMatrix;
```

Użyte wzorce:

- **Object pattern** - w celu organizacji danych oraz zarządzania pamięcią
- **Singleton pattern** - w celu zapewnienia istnienia tylko jednej instancji obiektu oraz niedopuszczenia do błędów wynikających do jednoczesnego dostępu do pinów kontrolera
- **Opaque pattern** - w celu ukrycia zawartości obiektu przed użytkownikiem oraz niedopuszczenia do edycji danych inaczej niż przez metody obiektu

Obiekt klasy LedMatrix odpowiada za wyświetlanie danych na fizycznej matrycy LED. Składa się on jedynie z mutex'u zabezpieczającego pracę wielowątkową oraz dwóch buforów przyjmujących dane gotowe do wyświetlenia.

W celu zapewnienia szybkiego odświeżania wyświetlacza funkcja odpowiadająca za przeniesienie danych z pamięci do fizycznych pinów musi zajmować możliwie jak najmniej czasu. Dlatego implementacja modułu obsługującego matrycę zajmuje się jedynie wyświetlaniem przetrzymywanych danych, które są dostarczone w formacie odpowiednich stanów rejestru BSRR.

Listing 12: Interface klasy LedMatrix

```
int LedMatrix_Init();

void LedMatrix_DeInit();

void LedMatrix_DisplayFrame();

void LedMatrix_ChangeFrame(Frame newFrame);
```

- **LedMatrix_Init** - alokuje pamięć dla obiektu, tworzy mutex
- **LedMatrix_DeInit** - niszczy obiekt, zwalnia pamięć
- **LedMatrix_DisplayFrame** - wyświetla dane obecnie wczytanej klatki
- **LedMatrix_ChangeFrame** - podmienia dane drugiego buforu na nowe

Główną funkcją modułu jest LedMatrix_DisplayFrame(),

Listing 13: LedMatrix_DisplayFrame

```
static void LedMatrix_LoadBuffer()
{
    xSemaphoreTake(_singleton.Mutex, portMAX_DELAY);

    if(_singleton.Buffer != NULL){
        memcpy(_singleton.DisplayData, _singleton.Buffer, sizeof(uint32_t) *
            ADDRESABLE_PIXELS * BCM_BIT_DEPTH);
        Frame_Delete(_singleton.Buffer);
        _singleton.Buffer = NULL;
    }
```

```

    xSemaphoreGive(_singleton.Mutex);
}

/....

void LedMatrix_DisplayFrame()
{
    // load second buffer if isnt NULL
    LedMatrix_LoadBuffer();

    uint16_t used_pixels_1 = 0;
    uint16_t used_pixels_2 = 0;

    // reset register row and colors value
    GPIOC->BSRR = ((A_Pin|B_Pin|C_Pin|D_Pin|COLOR_PINS) << 16)|OE_Pin;

    for(uint8_t row = 0; row < ADDRESSABLE_ROWS; row++)
    {
        used_pixels_1 = row * COLUMNS * BCM_BIT_DEPTH;
        for(uint8_t BCM_bit = 0; BCM_bit < BCM_BIT_DEPTH; BCM_bit++)
        {
            LED_stop_display();

            used_pixels_2 = BCM_bit * COLUMNS;

            HAL_DMA_Start_IT(   &hdma_tim8_ch1,
                                (uint32_t)&(_singleton.DisplayData[ used_pixels_1 +
                                    used_pixels_2 ]),
                                (uint32_t)&(GPIOC->BSRR),
                                COLUMNS
                            );

            //wait for task notify
            xTaskToNotify = xTaskGetCurrentTaskHandle();

            //timer update event
            TIM8->EGR = TIM_EGR_UG;
            //timer start in OPM mode + start counter,
            //starts DMA transfer
            TIM8->CR1 = TIM_CR1_CEN | TIM_CR1_OPM;

            //task notify wait
            ulTaskNotifyTake( pdTRUE, portMAX_DELAY );

            LED_start_display();
            BCM_weight(BCM_delay[BCM_bit]);
            LED_stop_display();
        }
    }
}

// ...

// DMA callback - transfer done
void DMA_CB(DMA_HandleTypeDef *hdma){
  GPIOC->BSRR = LAT_Pin;
  //__ISB
  GPIOC->BSRR = (LAT_Pin) << 16;

  // task notify
```

```
  vTaskNotifyGiveFromISR(xTaskToNotify,&xHigherPriorityTaskWoken );
  xTaskToNotify = NULL;
  portYIELD_FROM_ISR( xHigherPriorityTaskWoken);
}
```

To ta funkcja odpowiada za transfer danych do rejestru BSRR i tym samym za wyświetlanie danych na matrycy. Na początku sprawdzana jest zawartość drugiego bufforu, jeżeli taka istnieje jest ona ładowana do buforu wyświetlanego, następnie za pomocą rejestru BSRR wszystkie piny sterujące matrycą są ustawiane na stan niski (poza OE, który używa odwrotnej logiki). Później w pętli ładowane i wyzwalane są transfery DMA zgrane z Timerem w trybie One Pulse tworzącym sygnał na pinie CLK 64 razy (wartość COLUMNS), każdy transfer ładuje jeden rząd danych, odpowiednio przesuwając rejestr przesuwny pinem CLK. Po zakończonym transferze tworzone jest przerwanie które obsługując pin LAT zatrzaskuje stan rejestru przesuwnego matrycy tym samym zapisując dane dla rzędu pixeli. Główna pętla funkcji czeka na Notify z obsługi przerwania (potrzebne w celu zgrania DMA z procesorem). Po otrzymaniu komunikatu że transfer DMA się zakończył rząd pixeli jest włączany na odpowiedni czas wynikający z modulacji BCM. Każdy rząd jest wyświetlany 4 razy z celu utworzenia koloru. Następnie cały proces jest powtarzany dla każdego rzędu tym samym wyświetlając jedną klatkę w 12 bitowym kolorze.

4.4 Image

Listing 14: strukturta Image

```
typedef struct{
    ImageBackground * Background;
    ImageMovingItem * MovingItem;
}Image;
```

Struktura Image służy do organizacji klas ImageBackground oraz ImageMovingItem w jeden obraz. Taki obraz jest gotowy do umieszczenia na płótnie jakim jest klasa Canvas

4.4.1 ImageBackground

Listing 15: klasa ImageBackground

```
typedef struct{
    // wska nik na tablice wska nik w szeroko ci matrycy
    const uint8_t (*BackgroundMask)[COLUMNS];
    // kolor t a
    COLOR Color;
}ImageBackground;

void ImageBackground_Init(ImageBackground * self, const uint8_t (*newBackgroundMask)[
    COLUMNS], COLOR newColor);

void ImageBackground_SetColor(ImageBackground * self, COLOR newColor);

void ImageBackground_SetNewBackgroundData(ImageBackground * self, const uint8_t (*
    newBackgroundMask)[COLUMNS]);

//...

static const uint8_t simpleBG [ROWS][COLUMNS] = {
...
}
```

Użyte wzorce:

- **Object pattern** - w celu organizacji danych

Klasa ImageBackground trzyma wskaźnik do stałego "obrazu", czyli tablicy dwuwymiarowej wielkości matrycy LED, który składa się na stałą część obrazu, oraz kolor w jakim ma zostać wyświetlony obraz.

Obrazy znane są już na poziomie kompilacji i trzymane w pamięci flash, przykładem jest simpleBG na załączonym elemencie kodu.

- **ImageBackground_Init** - ładuje dane do obiektu
- **ImageBackground_SetColor** - zmienia kolor w którym wyświetlane ma być tło
- **ImageBackground_SetNewBackgroundData** - zmienia wskaźnik do stałego "obrazu"

4.4.2 ImageMovingItem

Listing 16: klasa ImageMovingItem

```
// stuktura opisuj ca maksymalne przsuni cie obiektu w dw ch osiach
typedef struct{
    // maksymalne przesuni cie w pikselach na osi X
    uint16_t maxOffsetX;
    // maksymalne przesuni cie w pikselach na osi Y
    uint16_t maxOffsetY;
}MoveRange;

// struktura opisuj ca zakres danych wej ciowych opisuj cych po o enie
typedef struct{
    // warto    dla minimalnego wychylenia X
    float minX;
    // warto    dla maksymalnego wychylenia X
    float maxX;
    // warto    dla minimalnego wychylenia Y
    float minY;
    // warto    dla maksymalnego wychylenia Y
    float maxY;
}PositionDataRange;

// obiekt dynamicznego elementu orazu
typedef struct{
    // wska nik na dane maski obiektu w zerowym wychyleniu
    const uint8_t (*ItemMask)[COLUMNS];
    // kolor elementu
    COLOR Color;
    // maksymalne wychylenia
    MoveRange MoveLimts;
    // zakres danych wej ciowych
    PositionDataRange InputRange;
    // obecne wychylenie obiektu w osi X
    uint16_t OffsetX;
    // obecne wychylenie obiektu w osi Y
    uint16_t OffsetY;
    // warto    danych wej ciowych potrzebna do przesuni cia elementu o jeden
        piksel w osi X
    float ShiftUnitX;
    // warto    danych wej ciowych potrzebna do przesuni cia elementu o jeden
        piksel w osi Y
    float ShiftUnitY;
}ImageMovingItem;

```

```
void ImageMovingItem_Init(ImageMovingItem * self, const uint8_t (*newItemMask)[
    COLUMNS], MoveRange newMoveLimits, PositionDataRange newInputRange, COLOR
    newColor);

void ImageMovingItem_SetPosition(ImageMovingItem * self, float inputX, float inputY);

void ImageMovingItem_SetNewItemData(ImageMovingItem * self, const uint8_t (*
    newItemMask)[COLUMNS]);

//...

static const uint8_t simpleEyes [ROWS][COLUMNS]= {
...
}
```

Użyte wzorce:

- **Object pattern** - w celu organizacji danych

Klasa ImageMovingItem trzyma wskaźnik do stałego "obrazu", czyli tablicy dwuwymiarowej wielkości matrycy LED, który składa się na dynamiczną część obrazu. Obraz ten jest zapisany w postaci zerowego wychylenia.

Klasa ImageMovingItem jest dynamicznym elementem obrazu który zmienia swoje położenie w zależności od podanych danych wejściowych.

Obrazy znane są już na poziomie kompilacji i trzymane w pamięci flash, przykładem jest simpleEyes na załączonym elemencie kodu.

- **ImageMovingItem_Init** - ładuje dane do obiektu. Przy inicjalizacji obiektu definiowane są MoveLimits, które przyjmują wartości dla osi X i Y opisujące o ile pikseli dany obraz może przesunąć się na wskazanych osiach. Definiowane również są zakresy danych sterujących dane wychylenia w zmiennej InputRange. Zakres wartości wejściowych jest mapowany na ilość możliwych położeń w dwóch osiach i obliczana jest jednostka ShiftUnitX oraz ShiftUnitY, która jest potrzebna do przesunięcia obiektu o jeden piksel na osi.
- **ImageMovingItem_SetPosition** - przyjmuje dane wejściowe inputX i inputY, które są następnie dzielone przez ShiftUnitX oraz ShiftUnitY tym samym wyliczając przesunięcie obrazu w pikselach na danych osiach
- **ImageMovingItem_SetNewItemData** - zmienia wskaźnik do przesuwanego "obrazu"

4.5 Font

Listing 17: klasa Font

```
// struktura na dane kszta tu czcionki
typedef struct{
    uint8_t  shape[5];
}charShape;

// dost pne generatory czcionki
typedef enum{
    BASIC
}FONT_TYPE;

void Font_GetCharShape(FONT_TYPE fontType, char character, charShape * retShape);

void Font_GetShapeHeight(FONT_TYPE fontType, uint8_t * retHeight);

void Font_GetShapeWidth(FONT_TYPE fontType, uint8_t * retWidth);


// struktura opisuj ca generator
typedef struct{
```

```
    charShape letters[26];
    charShape numbers[10];
    charShape space;
    charShape err;
    uint8_t shapeWidth;
    uint8_t shapeHeight;
}FontData;

// biblioteka generator w
static const FontData Fonts[] = {
    basicFont
};

// przyk adowy znak generatora
static const charShape s9 = {
    .shape = {
        0b1110,
        0b1010,
        0b1110,
        0b0010,
        0b1110
    }
};
```

Użyte wzorce:

- **Object pattern** - w celu organizacji danych

Klasa Font tworzy stałe obiekty w pamięci do których dostęp jest możliwy poprzez metody klasy w których dokładny obiekt specyfikowany jest parametrem fontType. Dzięki temu dostęp do obiektów przez użytkownika jest ograniczony, a wszystkie obiekty opisane przez FontData (generatory czcionki) muszą być znane na poziomie kompilacji.

Tak jak obiekty klasy Image, klasa Font służy do nakładania na "płótno"jakim jest klasa Canvas, definiuje ona zestaw kształtów trzymanych w stałych typu charShape, które są maską bitową.

- **Font_GetCharShape** - kopiuje kształt odpowiadający wpisanemu znakowi character pod wskaźnik retShape z wybranego generatora fontType
- **Font_GetShapeHeight** - wpisuje wysokość znaku danego generatora pod wskaźnik retHeight
- **Font_GetShapeWidth** - wpisuje wysokość znaku danego generatora pod wskaźnik retWidth

4.6 Canvas

Listing 18: klasa Canvas

```
typedef struct{
    SemaphoreHandle_t Mutex;
    uint16_t ** RGB_data;
}Canvas;

int  Canvas_Create(Canvas * self);

void Canvas_Reset(Canvas * self);

void Canvas_Destroy(Canvas * self);

int Canvas_PutImage(Canvas * self, Image * newImage);

int Canvas_PutTextLine(Canvas * self, FONT_TYPE font, TEXT_LINE line, const char *
    text, int16_t pixel_shift, COLOR color);

Frame Canvas_GenerateFrame(Canvas * self);
```

Użyte wzorce:

- **Object pattern** - w celu organizacji danych oraz zarządzania pamięcią

Klasa Canvas służy jako płótno na które nakładane są dane takie jak tekst lub obrazy, które później jest tłumaczone na format czytelny dla LedMatrix czyli Frame. Na płótno dane nakładane są w formacie RGB.

- **Canvas_Create** - alokuje pamięć dla obiektu, tworzy mutex
- **Canvas_Reset** - zeruje całe płótno
- **Canvas_Destroy** - niszczy obiekt i zwalnia pamięć
- **Canvas_PutImage** - przyjmuje wskaźnik na obiekt Image, który następnie nakłada na płótno
- **Canvas_PutTextLine** - wstawia tekst wygenerowany przez wskazaną czcionkę do odpowiedniej lini tekstu na płótnie, przesunięty o pixel_shift pixeli względem lewej krawędzi matrycy (pixel_shift > 0 przesunięcie w prawo, pixel_shift < 0 przesunięcie w lewo), w kolorze color
- **Canvas_GenerateFrame** - Zwraca obiekt Frame wygenerowany na podstawie obecnej zawartości płótna

Płótno o którym mowa jest tablicą dwuwymiarową o wymiarach matrycy LED, przy metodach PutImage oraz PutTextLine brane są dane opisujące kształty (charShape, ItemMask, BackgroundMask) w postaci maski bitowej, następnie odpowiednio przesuwane oraz wkładane do tablicy (płótna) w podanych kolorach. Jako że obiekty Image przekazywane są jako wskaźniki, edycja przekazanych obiektów wpływa również na trzymany przez Canvas obraz. Jednak po takiej zmianie niezbędne jest ponowne nałożenie na płótno oraz wygenerowanie klatki.

Generowanie klatki polega na przetłumaczeniu kolorów RGB każdego pixelu na stany rejestru BSRR dla każdego z nich.

5 Implementacja modułu pamięci SD

Moduł odpowiada za pełny łańcuch obsługi karty SD: od komunikacji sprzętowej przez system plików po czytanie i mapowanie animacji. Architektura oparta jest na warstwowych abstrakcjach: sterownik sprzętu, system plików, czytnik animacji oraz mapowanie współrzędnych na ramki.

5.1 Warstwa sprzętowa: SD_SPI Driver

Moduł `sd_spi.c/h` implementuje komunikację z kartą SD poprzez interfejs SPI (Serial Peripheral Interface). Stanowi najniższą warstwę abstrakcji sprzętu.

5.1.1 Struktury danych

Listing 19: Struktura informacyjna karty SD

```
typedef struct {
    SD_CardType_t type;          // Typ karty (V1, V2, SDHC)
    uint32_t capacity;           // Pojemno   w blokach 512 bajt w
    bool initialized;            // Flaga inicjalizacji
} SD_CardInfo_t;
```

Typ wyliczeniowy `SD_Status_t` definiuje możliwe statusy operacji: `SD_OK`, `SD_ERROR`, `SD_TIMEOUT`, `SD_NOT_INITIALIZED`.

5.1.2 Zastosowane wzorce projektowe

- **Hardware Abstraction Layer (HAL):** Wszystkie operacje SPI są zahermetyzowane wewnątrz funkcji statycznych (`SD_SendByte`, `SD_SendCommand`). Kod użytkownika nie ma bezpośredniego dostępu do `hspi2`.
- **State Machine (Automat stanów):** Inicjalizacja karty realizuje sekwencję stanów: reset → identyfikacja → negocjacja napięcia → uruchomienie trybu bitowego.
- **Resource Management:** Funkcje zawsze przywracają pin CS (Chip Select) do stanu wysokiego, zapobiegając blokowaniu magistrali SPI.

5.1.3 Funkcje publiczne

SD_Init()

- Wysyła 80 taktów zegarowych by obudzić kartę.
- Wykonuje reset (CMD0) i wysyła warunki zasilania (CMD8).
- Negocjuje typ karty (SDSC vs SDHC) za pomocą ACMD41.
- Konfiguruje szybkość SPI i zapisuje informacje w strukturze `card_info`.
- **Zwraca:** `SD_OK` lub `SD_TIMEOUT` w przypadku błędu.

SD_ReadBlock(block_addr, buffer) / SD_ReadMultipleBlocks(block_addr, buffer, count)

- Wysyła komendę `CMD17` (READ_SINGLE_BLOCK).
- Czeka na token danych (0xFE).
- Odczytuje 512 bajtów danych i 2 bajty CRC (ignorowane).
- Dla SDSC karty konwertuje adres: `block_addr` → `block_addr * 512` (adresowanie bajtowe).
- Dla SDHC karty adres jest już w blokach.

SD_WriteBlock(block_addr, buffer)

- Wysyła komendę `CMD24` (WRITE_BLOCK).

- Wysyła token 0xFE, następnie 512 bajtów danych i 2 bajty CRC.
- Czeka na potwierdzenie zapisu (bity 0-4 = 0x05).
- Czeka aż karta zakończy fizyczny zapis (busy wait).

SD_GetCardInfo(), SD_IsInitialized()

- Gettery zapewniające odczyt informacji o karcie.
- Brak modyfikacji stanu karty – bezpieczne dla wielowątkowego dostępu (tylko odczyt).

5.2 Warstwa systemu plików: FatFS Integration

Moduł `FATFS/Target/user_diskio.c` łączy sterownik SD z biblioteką FatFS poprzez implementację interfejsu `Diskio`.

5.2.1 Architektura

FatFS wymaga dostarczenia sześciu funkcji zwrotnych (callback'ów):

- `USER_initialize()` – inicjalizacja dysku.
- `USER_status()` – sprawdzenie stanu gotowości.
- `USER_read()` – odczyt sektorów.
- `USER_write()` – zapis sektorów (opcjonalnie).
- `USER_ioctl()` – operacje kontrolne (opcjonalnie).

Listing 20: Schemat inicjalizacji FatFS

```
// W StartDefaultTask():
FRESULT fres = f_mount(&USERFatFS, USERPath, 1);  // Force mount
if (fres == FR_OK) {
    // System plik w gotowy
}
```

Ważne konfiguracje (ffconf.h):

- `FF_USE_LFN = 0` – Długie nazwy plików wyłączone. Wymusza format 8.3 (np. `TEST~1.ANI`).
- `FF_FS_REENTRANT = 0` – Brak thread-safe'u. Zapewnia lepszą wydajność dla single-threaded kontekstu.
- `FF_FS_READONLY = 0` – Zapis włączony (opcja dla przyszłych rozszerzeń).

Krytyczne szczegóły implementacji:
Funkcja `USER_initialize()` weryfikuje inicjalizację karty SD:

Listing 21: Inicjalizacja dysku

```
DSTATUS USER_initialize(BYTE pdrv) {
    if (pdrv != 0) return STA_NOINIT;

    SD_Status_t status = SD_Init();  // Inicjalizacja sterownika
    if (status == SD_OK) {
        Stat &= ~STA_NOINIT;  // Dysk gotowy
        return Stat;
    }
    return STA_NOINIT;
}
```

Funkcja `USER_read()` mapuje logiczne sektory FatFS na fizyczne bloki karty:

Listing 22: Odczyt sektorów

```
DRESULT USER_read(BYTE pdrv, BYTE *buff, DWORD sector, UINT count) {
    if (pdrv != 0 || Stat & STA_NOINIT) return RES_NOTRDY;

    SD_Status_t status = SD_ReadMultipleBlocks(sector, buff, count);
    return (status == SD_OK) ? RES_OK : RES_ERROR;
}
```

5.2.2 Uwaga o osDelay()

Krytyczne odkrycie: Funkcja `osDelay()` zawieszała się w kontekście tasku FreeRTOS bezpośrednio po inicjalizacji SD. Przyczyna: `osDelay()` wykonuje przełączenie kontekstu (context switch), co może spowodować race condition w niedojrzałym systemie RTOS.

Rozwiązanie: Zastosowano busy-wait z instrukcjami NOP:

Listing 23: Busy-wait zamiast osDelay

```
// NIEW A CIWE:
osDelay(50);  // Mo e zawisn

// W A CIWE:
for (volatile uint32_t d = 0; d < 500000; d++) {
    __NOP();  // Op  nienie bez prze  czenia kontekstu
}
```

5.3 Warstwa czytnika animacji: AnimationReader (OOP w C)

Moduł `sd/Src/sd_animation_reader.c` implementuje czytnik ramek animacji z karty SD, wykorzystując interfejs oparty na wirtualnych tabelach metod (OOP w C).

5.3.1 Struktury danych

Listing 24: Struktura bazowa interfejsu

```
typedef struct {
    uint32_t frame_count;       // Liczba ramek
    uint32_t fps;               // Klatki na sekund
    uint32_t width, height;     // Wymiary ramki
    char name[32];              // Nazwa animacji
} AnimationMetadata_t;

typedef struct AnimationReader {
    const AnimationReaderVTable_t* vtable;  // Wska nik na metody
    void* private_data;                     // Dane specificzne dla implementacji
    bool initialized;                       // Stan inicjalizacji
} AnimationReader_t;
```

Listing 25: Struktura konkretnej implementacji dla SD

```
typedef struct {
    FIL file;                       // Uchwyt do otwartego pliku
    AnimationMetadata_t metadata;   // Metadane animacji
    char file_path[64];             //    cieka    do pliku
} SDAnimationReaderData_t;

typedef struct {
    AnimationReader_t base;         //  C z       publiczna (interfejs)
    SDAnimationReaderData_t data;   //  C z       prywatna (implementacja)
} SDAnimationReader_t;
```

5.3.2 Format pliku animacji

Plik animacji (`.ANI`) zawiera nagłówek 64-bajtowy oraz ramki:

Listing 26: Format nagłówka animacji

```
struct AnimationHeader {
    uint32_t magic;           // "ANIM" (0x4D494E41)
    uint32_t version;         // Wersja (1)
    uint32_t frame_count;     // Liczba ramek
    uint32_t fps;             // Klatki na sekund
    uint32_t width;           // Szeroko    (dla przysz o ci)
    uint32_t height;          // Wysoko    (dla przysz o ci)
    char name[32];            // Nazwa animacji
    uint8_t reserved[8];      // Zarezerwowane
};
// Ramki: ka da 16384 bajt w (4096 x uint32_t)
```

5.3.3 Funkcje publiczne

SDAnimationReader_Create(reader)

- Konstruktor obiektu. Przypisuje wirtualną tabelę metod (`vtable`).
- Zeruje całą strukturę.
- Nie wykonuje operacji I/O.

AnimationReader_Init(reader, path)

- Otwiera plik animacji za pomocą FatFS (`f_open`).
- Odczytuje i waliduje nagłówek (sprawdza magic i version).
- Ekstrahuje metadane (frame_count, fps, nazwa).
- Zamyka plik (zostanie ponownie otwarty przy odczycie).
- **Zwraca:** `ANIM_OK` lub kod błędu (`ANIM_FILE_NOT_FOUND`, `ANIM_ERROR`).

AnimationReader_GetMetadata(reader, metadata)

- Getter zwracający metadane animacji (read-only).

AnimationReader_ReadFrame(reader, frame_index, buffer)

- Otwiera plik animacji.
- Oblicza pozycję ramki: `offset = 64 + (frame_index * 16384)`.
- Przeskakuje do pozycji (`f_lseek`).
- Odczytuje 16384 bajty ramki do bufora.
- Zamyka plik.
- **Zwraca:** `ANIM_OK` lub kod błędu.

  Operacja jest blokująca – czeka na zakończenie odczytu z karty SD (około 50–100 ms dla ramki 16 KB).

AnimationReader_ReadFrameRange(reader, start_frame, count, buffer)

- Optymizacja: odczytuje wiele ramek jednym ciągiem zamiast osobno.
- Przydatna do preloadingu kolejnych ramek.

5.3.4 Uwagi implementacyjne

- **Nie thread-safe:** Struktura `AnimationReader` zawiera wspólny uchwyt `FIL`. Jednoczesny dostęp z wielu wątków spowoduje uszkodzenie danych. W przyszłości: dodać mutex do struktury.
- **Alokacja pamięci:** Ramka 16 KB musi być alokowana przez użytkownika (`pvPortMalloc`). Rozmiar zdefiniowany jako `ANIMATION_FRAME_SIZE_BYTES`.

5.4 Warstwa mapowania: AnimationMapper

Moduł `sd/Src/animation_mapper.c` mapuje dane wejściowe (współrzędne 2D lub strefy czujnika) na indeksy ramek w siatce 8×8.

5.4.1 Koncepcja siatki animacji

Animacja zawiera 64 ramki ułożone w siatce 8×8:

Listing 27: Mapowanie siatki na indeks liniowy

```
frame_index = x_grid * 8 + y_grid;
// gdzie x_grid, y_grid     [0, 7]
// Przyk ady:
// (0, 0)     0      (1, 0)     8        (7, 7)     63
```

5.4.2 Dwie metody mapowania

Metoda 1: Współrzędne 2D z floatów
Funkcja `AnimationMapper_GetFrameIndex(float x, float y)`:

- Klampuje wejście do zakresu $[0.0, 7.0]$.
- Aplikuje operację `floor()` aby zamienić float na indeks siatki.
- Oblicza indeks: `frame = floor(x) * 8 + floor(y)`.
- **Zwraca:** Indeks 0–63.

  Przykłady:

  - $x = 2.7, y = 1.3 \rightarrow \text{floor}(2.7) = 2, \text{floor}(1.3) = 1 \rightarrow 2 \cdot 8 + 1 = 17$
  - $x = 7.9, y = 7.9 \rightarrow$ klampuje do 7.0, 7.0 $\rightarrow$ 63

Metoda 2: Strefy czujnika (0–63)
Funkcja `AnimationMapper_LoadFrameByZone(reader, zone, buffer)`:

- Bezpośrednie mapowanie: `zone` = `frame_index`.
- Klampuje strefę do 0–63.
- Idealne dla czujników toF (TOF/PIR), które natively zwracają indeks strefy.
- **Operacja:** Deleguje do `AnimationReader_ReadFrame(reader, zone, buffer)`.

Metoda 3: Hybrydowa (float współrzędne)
Funkcja `AnimationMapper_LoadFrame(reader, x, y, buffer)`:

- Konwertuje float'owe współrzędne na indeks za pomocą `GetFrameIndex`.
- Odczytuje ramkę via `AnimationReader_ReadFrame`.
- Zwraca `AnimationStatus_t`.

5.4.3 Diagram przepływu

Listing 28: Typowy przepływ w tasku

```
// Inicjalizacja (jednokrotnie)
SDAnimationReader_t reader;
SDAnimationReader_Create(&reader);
AnimationReader_Init(&reader.base, "/REG.ANI");

uint32_t* frame_buf = pvPortMalloc(ANIMATION_FRAME_SIZE_BYTES);

// P tla  g   wna (powtarzaj ~30 FPS)
for(;;) {
    float x = ObjectTracker_GetX(&tracker);   // 0.0  7  .0
    float y = ObjectTracker_GetY(&tracker);   // 0.0  7  .0

    // Mapowanie i odczyt
    AnimationStatus_t st = AnimationMapper_LoadFrame(
        &reader.base, x, y, frame_buf
    );

    if (st == ANIM_OK) {
        // frame_buf zawiera 16KB danych ramki
        DisplayFrame(frame_buf);  // Twoja funkcja wy wietlenia
    }

    osDelay(33);  // ~30 FPS
}
```

5.5 Integracja całości w FreeRTOS

5.5.1 Inicjalizacja

Listing 29: Pełny kod inicjalizacji w StartDefaultTask

```
void StartDefaultTask(void *argument)
{
    /* USER CODE BEGIN StartDefaultTask */

    // 1. Inicjalizacja SD (sprz t)
    printf("[RTOS] Initializing SD card...\r\n");
    SD_Status_t sd_status = SD_Init();
    if (sd_status != SD_OK) {
        printf("[ERROR] SD init failed\r\n");
        for(;;) osDelay(1000);
    }

    // 2. Busy-wait (zamiast osDelay)
    for (volatile uint32_t d = 0; d < 500000; d++) { __NOP(); }

    // 3. Montowanie FatFS
    printf("[RTOS] Mounting FatFS...\r\n");
    FRESULT fres = f_mount(&USERFatFS, USERPath, 1);
    if (fres != FR_OK) {
        printf("[ERROR] FatFS mount failed: %d\r\n", fres);
        for(;;) osDelay(1000);
    }

    // 4. Inicjalizacja czytnika animacji
    printf("[RTOS] Opening animation file...\r\n");
```

```
    SDAnimationReader_t reader;
    SDAnimationReader_Create(&reader);

    AnimationStatus_t anim_st = AnimationReader_Init(
        &reader.base, "/REG.ANI"  // 8.3 format
    );
    if (anim_st != ANIM_OK) {
        printf("[ERROR] Animation open failed\r\n");
        for(;;) osDelay(1000);
    }

    // 5. Alokacja bufora ramki (16 KB)
    uint32_t* frame_buf = pvPortMalloc(ANIMATION_FRAME_SIZE_BYTES);
    if (!frame_buf) {
        printf("[ERROR] Frame buffer allocation failed\r\n");
        for(;;) osDelay(1000);
    }

    printf("[RTOS] Animation system ready\r\n");

    // 6. P tla g  wna
    for(;;) {
        float x = ObjectTracker_GetX(&myTracker);
        float y = ObjectTracker_GetY(&myTracker);

        AnimationStatus_t st = AnimationMapper_LoadFrame(
            &reader.base, x, y, frame_buf
        );

        if (st == ANIM_OK) {
            DisplayFrame(frame_buf);  // Wy wietl na matrycy LED
        }

        osDelay(33);  // ~30 FPS
    }

    /* USER CODE END StartDefaultTask */
}
```

5.5.2 Konfiguracja zasobów

- **FreeRTOS Heap:** `configTOTAL_HEAP_SIZE` 24576 bajtów (24 KB) – dla bufora ramki (16 KB) + metadane.
- **SPI2 Configuration:**
  - PB12 (CS) – GPIO output
  - PB13 (SCK) – AF (Alternate Function)
  - PB14 (MISO) – AF
  - PB15 (MOSI) – AF
  - Prescaler: 16 lub 32 (prędkość 1–2 MHz)
- **FatFS:** `FF_USE_LFN = 0` – wymusza nazwy 8.3 (np. `REG.ANI`).

5.5.3 Kodowanie błędów

Listing 30: Kody statusu animacji

```
typedef enum {
    ANIM_OK = 0,                    // Sukces
    ANIM_ERROR = 1,                 // Generyczny  b    d
    ANIM_FILE_NOT_FOUND = 2,        // Plik nie istnieje
    ANIM_INVALID_FRAME = 3,         // Indeks ramki poza zakresem
    ANIM_READ_ERROR = 4,            //  B    d  odczytu (FatFS/SD)
    ANIM_NOT_INITIALIZED = 5        // Reader nie zainicjalizowany
} AnimationStatus_t;
```

5.6 Podsumowanie architektury

Architektura jest warstwowa i modułowa:

- **Warstwa 1 (Sprzęt):** `sd_spi.c` – sterowanie SPI, blokowy dostęp do karty.
- **Warstwa 2 (System plików):** FatFS + `user_diskio.c` – abstrakcja plików.
- **Warstwa 3 (Animacja):** `sd_animation_reader.c` – czytnik ramek, format .ANI.
- **Warstwa 4 (Logika):** `animation_mapper.c` – mapowanie współrzędnych ramki.
- **Warstwa 5 (Aplikacja):** FreeRTOS Task – integracja z czujnikami i wyświetlaczem.

Każda warstwa ma jasno zdefiniowany interfejs, co ułatwia testowanie, modyfikacje i ponowne użycie.